Exact Evaluation of QOp with Cyclotomic Arithmetic

Mathematical foundations, two evaluators, and exact MPS projects

QPP / HQP · 17 October 2026

Abstract

Quantum Programming Playground (QPP) represents quantum operators as symbolic programs whose angles are rational multiples of π. Their floating-point evaluation can be replaced by exact arithmetic: every valid finite operator expression has its entries in a computable cyclotomic field. We introduce the language and the necessary algebra, derive this closure result, and develop dense matrix and direct statevector evaluation as two implementations of the same semantics. A worked two-qubit example connects syntax, matrices, amplitudes, and probabilities. We then explain why matrix product states can remain cyclotomic even when their singular value decompositions cannot, and develop exact rank factorization into a four-week HQP project and an MSc thesis proposal. An independent student project develops the prototype scalar arithmetic into a more efficient implementation. The accompanying prototype implements scalar arithmetic and both backends, with exact expected-value and randomized tests. The MPS work remains proposed; no comparative performance claim is made.

1 Why exact evaluation, and what must change?

Compiler transformations ask algebraic questions: does a circuit followed by its adjoint give identity, does a rewrite preserve every amplitude, and is an apparently zero entry actually zero? Floating-point evaluation answers within a tolerance. Exact evaluation makes equality and cancellation mathematical decisions.

QPP is a Haskell framework whose symbolic operator language, `QOp`, has several evaluators. Its `Rational` angles must become exact *complex coefficients* during evaluation.

Rationals alone cannot represent a Hadamard coefficient $1/\sqrt{2}$ or a nonreal phase. Cyclotomic fields contain both and are closed under the operations used to compose gates. Each finite expression uses a finite-degree number field; its degree and coefficient sizes determine cost.

The dense evaluator is a small-circuit oracle; direct statevector evaluation applies operators without constructing their matrices. Both support later MPS work.

Scope and reading guide. We assume basic linear algebra and familiarity with qubits, but no knowledge of QPP or algebraic number computation. Two explicitly marked optional arguments use Galois theory to prove limitations of SVD and normalization; their conclusions suffice for the evaluator design. Sections 2–5 introduce the language, semantics, and exactness theorem. Section 6 works through two examples; Sections 7–9 specify implementation and validation. The final sections develop MPS and the student projects in tensor networks and scalar arithmetic. Initial execution supports unitary operators on supplied cyclotomic vectors. Measurement, initialization by measurement and reset, and arbitrary normalization are separate extensions.

2 The operator language and its mathematical objects

A `QOp` constructor builds an abstract syntax tree and performs no numerical evaluation. The full datatype appears below, after the conventions needed to read it.

Terms, operators, and vectors. Constructor names appear in monospace; u, t are terms. For a valid term u, its width n_u is the number of qubits on which it acts. Its denotation $[\![u]\!]$ is a linear operator on $\mathbb{C}^{2^{n_u}}$, represented in the computational basis by a $2^{n_u} \times 2^{n_u}$ matrix. The notation $[\![u]\!]\, v$ is the action of this same operator on a column vector v. We keep these brackets for both evaluators. Symbols such as $A = [\![u]\!]$ name mathematical matrices, not syntax.

The ordered basis is $|b_0 \cdots b_{n-1}\rangle$, with $b_j \in \{0, 1\}$ and index $\sum_j b_j 2^{n-1-j}$. Thus wire zero is the most significant bit. For two qubits the vector order is $(v_{00}, v_{01}, v_{10}, v_{11})^T$. The left tensor factor acts on the leading wires. A zero-qubit space has dimension one: `Id 0` denotes $[1]$, and `Phase q` denotes $[e^{i\pi q}]$.

Widths and validity. `Compose u t` applies t first and requires $n_u = n_t$. `Tensor u t` has width $n_u + n_t$. `DirectSum u t` requires $n_u = n_t$ and adds a selector qubit: its width is $n_u + 1$. Selector value zero chooses u and value one chooses t. In particular, `C u` has width $n_u + 1$ and selects between identity and u. Here `Nat` is represented by `Int`, so nonnegative widths must be checked explicitly. A wire permutation must be a bijection of its local indices.

A rotation `R u q` has width n_u. For $q \neq 0$, its axis must denote a Hermitian unitary matrix: $A^\dagger = A$ and $A^\dagger A = I$. Consequently $A^2 = I$. This is stronger than merely being Hermitian and is what makes the finite rotation formula possible. At $q = 0$, any well-formed axis is accepted: $e^{0A} = I$. Its subterms are still checked, so a zero angle cannot conceal invalid syntax. The adjoint † means conjugate transpose.

```
data QOp
  = Id Nat                    -- identity on n qubits
  | Phase Rational            -- a scalar phase, on zero qubits
  | X | Y | Z | H | SX         -- fixed one-qubit gates
  | R QOp Rational            -- rotation about an operator axis
  | C QOp                     -- add a leading control qubit
  | Permute [Nat]             -- reorder the local wires
  | Tensor QOp QOp
  | DirectSum QOp QOp
  | Compose QOp QOp
  | Adjoint QOp
```

How code expressions are read. QPP also has infix notation: `u <.> t` is `Tensor u t`, `u <+> t` is `DirectSum u t`, and `u <> t` is `Compose u t`. We mostly use constructors to expose the structure. For example, `Compose (C X) (Tensor H (Id 1))` applies a Hadamard to the first qubit and then a controlled X. Structural equality of syntax trees is distinct from equality of their denotations: `Compose H H` and `Id 1` are different terms denoting the same matrix.

3 One operator semantics for both evaluators

We first define meaning independently of storage and algorithms. If $D_u = 2^{n_u}$, each row of the following table specifies the denotation of a constructor. Matrix multiplication implements composition; $\otimes$ is the Kronecker product and $\mathrm{diag}(A, B)$ is a block diagonal matrix. The rotation convention includes a half angle.

Term	Matrix denotation
`Id n`	I_{2^n}.
`Phase q`	$[e^{i\pi q}]$.
`X, Y, Z, H, SX`	The matrices in (1).
`R u q`	$\exp(-i\pi q \, [\![u]\!]/2)$.
`C u`	$\mathrm{diag}(I_{D_u}, [\![u]\!])$.
`Permute ks`	$P_{f(j),j} = 1$ and other entries zero; the bits of $f(j)$ are $b_{\mathtt{ks}[0]}, \ldots, b_{\mathtt{ks}[n-1]}$ if j has bits $b_0, \ldots, b_{n-1}$ and $n = \lvert\mathtt{ks}\rvert$.
`Tensor u t`	$[\![u]\!] \otimes [\![t]\!]$.
`DirectSum u t`	$\mathrm{diag}([\![u]\!], [\![t]\!])$, with $n_u = n_t$.
`Compose u t`	$[\![u]\!] \, [\![t]\!]$, with $n_u = n_t$.
`Adjoint u`	$[\![u]\!]^\dagger$.

$$X := [\![\mathtt{X}]\!] = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y := [\![\mathtt{Y}]\!] = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z := [\![\mathtt{Z}]\!] = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$
$$H := [\![\mathtt{H}]\!] = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \mathrm{SX} := [\![\mathtt{SX}]\!] = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}. \tag{1}$$

The chosen square root of X satisfies $\mathrm{SX}^2 = X$. It is unitary but not Hermitian, so `SX` is not a valid axis at a nonzero angle. The term `H`, in contrast, is a valid axis: rotations are not restricted to Pauli matrices.

Why global phase must be retained. For an isolated state, v and $e^{i\alpha}v$ give the same measurement probabilities. Inside an operator language, however, control converts this phase into a relative phase:

$$[\![\texttt{C (Phase q)}]\!] = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi q} \end{pmatrix}.$$

Thus equality includes phase: $[\![\texttt{C (R (Id 0) 2)}]\!] = Z$, not I_2. Writing $S = \mathrm{diag}(1, i)$ and $T = \mathrm{diag}(1, e^{i\pi/4})$, the rotation convention gives

$$[\![\texttt{R Z (1/2)}]\!] = e^{-i\pi/4}S, \qquad [\![\texttt{R Z (1/4)}]\!] = e^{-i\pi/8}T.$$

The terms `Phase (1/4) <.> R Z (1/2)` and `Phase (1/8) <.> R Z (1/4)` denote S and T exactly. These examples explain why a semantics suitable for compiler validation must preserve scalar phases even on zero qubits.

4 Cyclotomic arithmetic: what a coefficient contains

An exact coefficient is a complex number represented by rational coordinates in a specified basis. We distinguish the number z, a polynomial $p(x)$ representing it, and the tuple stored by the evaluator. Here x is a formal variable; ζ_N is an actual complex root of unity, and substitution gives $z = p(\zeta_N)$. Several polynomials can represent the same number. Reduction will select one of them uniquely. Section 9 adopts these coordinates as the baseline storage format, with an explicit field context.

4.1 A first example: the eighth roots of unity

Let $\zeta_8 = e^{2\pi i/8} = (1 + i)/\sqrt{2}$. Its powers satisfy $\zeta_8^2 = i$ and $\zeta_8^4 = -1$. The latter identity lets us replace any higher power by a rational multiple of a lower one; for example $\zeta_8^6 = -\zeta_8^2$ and $\zeta_8^9 = \zeta_8$. Every element of the field $K_8 = \mathbb{Q}(\zeta_8)$ has a unique expression

$$z = z_0 + z_1\zeta_8 + z_2\zeta_8^2 + z_3\zeta_8^3, \qquad z_j \in \mathbb{Q}.$$

Here $\mathbb{Q}(\zeta_8)$ means the smallest field containing the rationals and ζ_8: it is closed under addition, subtraction, multiplication, and division by nonzero elements. The four powers above form a basis over $\mathbb{Q}$, so z has coordinates (z_0, z_1, z_2, z_3) in that basis. The reason four coordinates suffice, even after division, is given below.

The Hadamard coefficient $h = 1/\sqrt{2}$ is the real part of ζ_8. Since $\zeta_8^{-1} = -\zeta_8^3$, it has the exact representation

$$h = \frac{\zeta_8 + \zeta_8^{-1}}{2} = \frac{\zeta_8 - \zeta_8^3}{2} \quad \longleftrightarrow \quad (0, 1/2, 0, -1/2).$$

Squaring this expression illustrates the arithmetic:

$$h^2 = \frac{\zeta_8^2 - 2\zeta_8^4 + \zeta_8^6}{4} = \frac{\zeta_8^2 + 2 - \zeta_8^2}{4} = \frac{1}{2}.$$

We multiplied symbolic expressions and used $\zeta_8^4 = -1$ to reduce the powers. Thus the cancellation needed to verify $H^2 = I$ uses rational arithmetic; it never evaluates a decimal approximation to $\sqrt{2}$.

4.2 Why finite coordinates exist and are unique

For any positive integer N, fix $\zeta_N = e^{2\pi i/N}$ and write $K_N = \mathbb{Q}(\zeta_N)$. This root is *primitive*: its smallest positive power equal to one is N. The primitive Nth roots are precisely ζ_N^k for $1 \le k \le N$ with $\gcd(k, N) = 1$. The *cyclotomic polynomial* $\Phi_N(x)$ has these roots, each once, and leading coefficient one. For example, the primitive eighth roots are the four odd powers of ζ_8, and $\Phi_8(x) = x^4 + 1$.

The key theorem is that Φ_N has rational coefficients and is irreducible over $\mathbb{Q}$: it has no factorization into two rational polynomials of positive degree [1]. Its degree is $d = \varphi(N)$, the number of integers in $\{1, \ldots, N\}$ coprime to N. It is the polynomial of least positive degree over $\mathbb{Q}$ that vanishes at ζ_N.

To reduce a polynomial $p(x)$, divide it by $\Phi_N(x)$:

$$p(x) = q(x)\Phi_N(x) + r(x), \qquad \deg r < d.$$

Evaluating at ζ_N gives $p(\zeta_N) = r(\zeta_N)$ because $\Phi_N(\zeta_N) = 0$. The coefficients of r are therefore sufficient. They are also unique: two different remainders with the same value would have a nonzero difference of degree below d vanishing at ζ_N, contradicting minimality.

So far this establishes unique coordinates for polynomial expressions. Within the fixed basis, equality is coordinatewise rational equality, and zero means all coordinates are zero. We next show that division also produces a polynomial expression, completing the field construction.

4.3 Arithmetic is polynomial arithmetic followed by reduction

Suppose $a = p(\zeta_N)$ and $b = q(\zeta_N)$, with p, q already reduced. For $a + b$, add their coefficient tuples. For ab, multiply p and q, then take the remainder on division by Φ_N. The computation of h^2 above is exactly this procedure for $\Phi_8 = x^4 + 1$.

Conjugation uses the fact that a root of unity lies on the unit circle. Since the coefficients of p are rational and hence real,

$$\overline{p(\zeta_N)} = p(\overline{\zeta_N}) = p(\zeta_N^{-1}) = p(\zeta_N^{N-1}).$$

Substitute x^{N-1} for x in p, then reduce as before. For instance, $i = \zeta_8^2$ becomes $\zeta_8^6 = -i$ under conjugation.

Division also stays in the same coordinate system. If $p \neq 0$ and $\deg p < d$, irreducibility implies $\gcd(p, \Phi_N) = 1$. The extended Euclidean algorithm on rational polynomials finds s, t with

$$s(x)p(x) + t(x)\Phi_N(x) = 1.$$

At $x = \zeta_N$, the second term vanishes, leaving $s(\zeta_N)p(\zeta_N) = 1$. Thus s, reduced modulo Φ_N, represents a^{-1}. Rational expressions therefore need no extra basis elements. As a small example, $h^2 = 1/2$ gives $h^{-1} = 2h$.

The polynomial values form a field containing $\mathbb{Q}$ and ζ_N, and every field containing these must contain those values. They are therefore exactly K_N. In the standard quotient notation,

$$\mathbb{Q}[x]/(\Phi_N(x)) \;\cong\; K_N, \qquad [p] \longmapsto p(\zeta_N). \tag{2}$$

Here $\mathbb{Q}[x]$ denotes rational polynomials, and $[p]$ identifies all polynomials differing from p by a multiple of Φ_N. Its unique reduced representative supplies the stored coordinates. A *cyclotomic number* belongs to some K_N.

4.4 Changing coordinates to combine fields

Tuples from different fields cannot simply be added entry by entry: their basis elements differ. For $a \in K_N$ and $b \in K_M$, choose $L = \operatorname{lcm}(N, M)$. Our fixed choice of complex roots gives

$$\zeta_N = \zeta_L^{L/N}, \qquad \zeta_M = \zeta_L^{L/M}.$$

Substitute these powers in the expressions for a and b and reduce modulo Φ_L. The resulting tuples share the basis $1, \zeta_L, \ldots, \zeta_L^{\varphi(L)-1}$ and can be combined or compared. These embeddings change the representation, not the complex number.

For example, a Hadamard coefficient in K_8 and a phase $z = \zeta_6$ can both be represented in K_{24}. Put $\xi = \zeta_{24}$; then $\zeta_8 = \xi^3$, $\zeta_6 = \xi^4$, and $\Phi_{24}(x) = x^8 - x^4 + 1$. Hence $\xi^9 = \xi^5 - \xi$, giving

$$h = \frac{\xi^3 - \xi^9}{2} = \frac{\xi + \xi^3 - \xi^5}{2}, \qquad z = \xi^4.$$

In the eight-coordinate basis, h is now $(0, 1/2, 0, 1/2, 0, -1/2, 0, 0)$; the value is still $1/\sqrt{2}$. Section 6 uses precisely this pair of coefficients.

For a whole program we can choose a sufficient common field once. Its index need not be minimal: $K_3 = K_6$, since $\zeta_6 = 1 + \zeta_3$ and $\zeta_3 = \zeta_6^2$. The field degree counts rational coordinates, while their numerator and denominator bit lengths determine the cost of rational arithmetic. Both sizes matter; exactness alone gives no efficiency guarantee.

5 Why every valid rational-angle term stays cyclotomic

Rational-angle phases are roots of unity; rotations need a further argument. For a nonzero rotation angle, the unitary axis must also be Hermitian. For $A = [\![u]\!]$ this implies $A^2 = I_{2^{n_u}}$. Therefore every even power of A is identity and every odd power is A. With $\theta = \pi q/2$, the matrix exponential separates into two scalar series:

$$\begin{aligned} e^{-i\theta A} &= \left(\sum_{k\geq 0} \frac{(-1)^k \theta^{2k}}{(2k)!}\right) I - i \left(\sum_{k\geq 0} \frac{(-1)^k \theta^{2k+1}}{(2k+1)!}\right) A \\ &= \cos\theta\, I - i \sin\theta\, A. \end{aligned} \tag{3}$$

For a reduced rational $q = a/b$, $b > 0$, set $w = e^{i\theta} = \zeta_{4b}^{a}$. Euler's identities now give the finite formula used by both evaluators:

$$\begin{aligned} [\![\mathtt{R}\ u\ q]\!] &= c_q\, [\![\mathtt{Id}\ n_u]\!] + d_q\, [\![u]\!], \\ c_q = \frac{w + w^{-1}}{2}, &\qquad d_q = \frac{w^{-1} - w}{2}. \end{aligned} \tag{4}$$

The series proves the identity; the evaluator never computes the series, trigonometric functions, or a numerical matrix exponential. At $q = 0$, $c_q = 1$ and $d_q = 0$, so the same formula holds without assuming a Hermitian axis; evaluation skips the axis action. For example, $q = 1/3$ gives $c_q = \sqrt{3}/2$ and $d_q = -i/2$. The formula applies to any valid composite axis, including `H` and `C X`, not only to single-qubit Pauli axes.

Choosing a sufficient field. A phase with parameter a/b is ζ_{2b}^{a}. The fixed gates need only i and $1/\sqrt{2}$, both in K_8. Write $s \preceq t$ when s is a syntactic subterm of t, including t. Collect all angle denominators, including those inside rotation axes:

$$N = \mathrm{lcm}(\{8\} \cup \{2b : \mathtt{Phase}\ (a/b) \preceq t\} \cup \{4b : \mathtt{R}\ u\ (a/b) \preceq t\})\,. \tag{5}$$

For instance, occurrences of `Phase (1/3)` and `R Z (1/5)` give the sufficient index $\mathrm{lcm}(8, 6, 20) = 120$, of degree $\varphi(120) = 32$. This index need not be the minimal conductor.

Proposition 1. *Every valid n-qubit term t denotes a unitary matrix with entries in $D_N = \mathbb{Z}[1/2, \zeta_N] \subset K_N$, for N from (5).*

Proof. The ring D_N consists of finite sums of powers of ζ_N with dyadic rational coefficients (integers divided by powers of two). The fixed gates and phases lie in this ring; (4) uses only its addition, multiplication, and halving operations. Composition, tensor and direct sums, control, permutations, and adjoints preserve the ring, since conjugation sends ζ_N to ζ_N^{-1}. Structural induction gives closure. The same induction proves unitarity: each primitive is unitary, and exponentiating a Hermitian axis with a purely imaginary parameter is unitary. A zero-angle rotation is identity. □

If an input vector has entries in K_M, application stays in $K_{\mathrm{lcm}(N,M)}$. General field inverses are unnecessary for unitary evaluation, but will be needed for MPS rank factorization.

6 Worked evaluation: a phase and a non-Pauli rotation

Consider a Bell state followed by a phase gate on its second qubit. The following Haskell definitions build the operator expression:

```
phaseGate q = C (Phase q)
bell = Compose (C X) (Tensor H (Id 1))
example = Compose (Tensor (Id 1) (phaseGate (1/3))) bell
```

Here `phaseGate q` acts on one qubit: it controls a zero-qubit scalar. Let $h = 1/\sqrt{2}$, $z = e^{i\pi/3} = \zeta_6$, and $B = [\![\mathtt{bell}]\!]$. In the basis $(00, 01, 10, 11)$, the matrix evaluator constructs

$$B = h\begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 1 & 0 & -1 & 0 \end{pmatrix}, \qquad U := [\![\mathtt{example}]\!] = h\begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & z & 0 & z \\ 0 & 1 & 0 & -1 \\ z & 0 & -z & 0 \end{pmatrix}.$$

The second matrix is obtained by multiplying B on the left by $I_2 \otimes \mathrm{diag}(1, z) = \mathrm{diag}(1, z, 1, z)$. The direct evaluator instead starts with a length-four vector and performs three local actions:

$$|00\rangle \longmapsto h(|00\rangle + |10\rangle) \longmapsto h(|00\rangle + |11\rangle) \longmapsto h(|00\rangle + z|11\rangle). \tag{6}$$

The last vector is exactly the first column of U. No 4×4 matrix is needed on this execution path.

The actual coefficient field. The expression contains Hadamards and `Phase (1/3)`, so the field rule gives $N = \mathrm{lcm}(8, 6) = 24$, with $\varphi(24) = 8$. Both backends can use the basis modulo $\Phi_{24}(x) = x^8 - x^4 + 1$. In that common field,

$$z = \zeta_{24}^4, \qquad h = \frac{\zeta_{24}^3 + \zeta_{24}^{-3}}{2}, \qquad \|U|00\rangle\|^2 = h^2(1 + \bar{z}z) = 1.$$

The angle $1/3$ determines ζ_6; the nonzero amplitudes are h and hz.

Observing the relative phase. Applying the inverse Bell circuit gives

$$B^\dagger U|00\rangle = \frac{1 + z}{2}|00\rangle + \frac{1 - z}{2}|10\rangle.$$

Since $z + \bar{z} = 1$, the squared amplitudes are $|1 + z|^2/4 = 3/4$ and $|1 - z|^2/4 = 1/4$. If the phase gate had been discarded, the result would instead be $|00\rangle$. This calculation uses exact inner products and shows why phase-sensitive equality matters. Computing these probabilities does not require implementing random sampling or state collapse.

A rotation about the Hadamard axis. Let `r = R H (1/3)`. Since $H^2 = I$, (4) gives $w = \zeta_{12}$, $c = \sqrt{3}/2$, and $d = -i/2$. The common field is K_{24}: in the notation of Section 4, $w = \xi^2$ and $i = \xi^6$. The matrix evaluator constructs

$$[\![r]\!] = cI_2 + dH = \begin{pmatrix} c + dh & dh \\ dh & c - dh \end{pmatrix}.$$

The direct evaluator computes $H|0\rangle = h(|0\rangle + |1\rangle)$ and combines it with the original input:

$$[\![r]\!]\,|0\rangle = c|0\rangle + d\,h(|0\rangle + |1\rangle) = \left(\frac{\sqrt{3}}{2} - \frac{i}{2\sqrt{2}}\right)|0\rangle - \frac{i}{2\sqrt{2}}|1\rangle.$$

These are the first-column entries of the matrix. Their squared magnitudes are $7/8$ and $1/8$, so the norm is already one. This example exercises the half angle, its sign, field promotion, and recursive axis application without calling trigonometric functions.

7 Constructing an exact dense matrix

The dense evaluator follows the table in Section 3. It makes every matrix entry available for inspection and serves as an oracle for direct and MPS evaluation.

Validation before arithmetic. First check widths and permutations throughout the tree, and prove that every nonzero-angle rotation axis is Hermitian. QPP's symbolic checker proves this for Hermitian primitives, suitable tensors and direct sums, control, adjoints, and certain products and conjugations. These are sufficient rules; a failure to prove Hermiticity does not establish non-Hermiticity. For instance, `R H q` is accepted; `R SX q` requires $q = 0$. A future fallback could construct an axis matrix exactly and test $A = A^\dagger$. It is not needed for the first implementation.

The exact wrapper produces a private tree with verified widths, so recursive evaluation does not repeatedly rediscover dimensions. Then collect the field index and enforce resource limits before allocating matrices. An external input vector may require a larger common field when the resulting operator is applied.

Storage and recursion. Store checked dimensions and a flat row-major array of cyclotomic values. In Haskell, a boxed vector holds these exact values, whose sizes can vary; the existing floating-point BLAS/LAPACK representation is unsuitable. The following schematic cases show where the scalar operations enter:

```
matrix (Compose u t) = multiply (matrix u) (matrix t)
matrix (Tensor u t)  = kronecker (matrix u) (matrix t)
matrix (Adjoint u)   = conjugateTranspose (matrix u)
matrix (R u q) =
  let a = matrix u
      (c, d) = rotationCoefficients q
  in addMatrix (scale c (identity (dimension a))) (scale d a)
```

In this sketch, `rotationCoefficients` implements (4) by root construction, exact halving, and multiplication. The zero-qubit phase is a 1×1 matrix throughout, including when it is placed under control or direct sum.

For compatible matrices and a column vector, use

$$(AB)_{ij} = \sum_k A_{ik} B_{kj}, \qquad (Av)_i = \sum_j A_{ij} v_j, \qquad (A \otimes B)_{ir+k,\, js+l} = A_{ij} B_{kl},$$

where B has shape $r \times s$. Strict loops accumulate exact scalars starting from zero. Skip an entry only after an exact zero test. Conjugate transpose exchanges indices and conjugates each coefficient; no magnitude threshold or numerical diagonalization is involved.

Cost and meaning of equality. For $D = 2^n$, storing the matrix costs $O(D^2)$ field elements, multiplication costs $O(D^3)$ field operations, and application costs $O(D^2)$. These are counts of operations in K_N, not bit complexities. Entrywise equality decides whether two evaluated operators are equal, including global phase. In a common polynomial basis, it reduces to rational equality.

8 Direct exact statevector evaluation

For a checked term u and compatible vector v, the direct evaluator computes $[\![u]\!]\, v$. The brackets denote the same operator as before. We traverse its syntax to update amplitudes without building the matrix, first embedding operator and input coefficients in a common cyclotomic field K.

A statevector is a length-2^n array. The computational basis vector $|j\rangle$ has a one at index j and zeros elsewhere; on zero qubits the sole basis vector is (1). We also admit unnormalized vectors: linearity requires the evaluator to work on arbitrary vectors, not only physical states of unit norm.

The recursive rules. Identity leaves its input unchanged, and `Phase q` maps the one-component vector (z) to $(e^{i\pi q} z)$. For a one-qubit primitive $G = (G_{ij})$, each relevant amplitude pair $(a, b)^T$ becomes $(G_{00}a + G_{01}b, G_{10}a + G_{11}b)^T$. The composite cases are

$$[\![\texttt{Compose}\ u\ t]\!]\, v = [\![u]\!]\, ([\![t]\!]\, v), \tag{7}$$

$$[\![\texttt{R}\ u\ q]\!]\, v = c_q v + d_q ([\![u]\!]\, v), \tag{8}$$

$$[\![\texttt{DirectSum}\ u\ t]\!]\, (v_0 \parallel v_1) = ([\![u]\!]\, v_0) \parallel ([\![t]\!]\, v_1), \tag{9}$$

$$[\![\texttt{C}\ u]\!]\, (v_0 \parallel v_1) = v_0 \parallel ([\![u]\!]\, v_1). \tag{10}$$

Here c_q, d_q are from (4), and $\parallel$ means concatenation. The two blocks correspond to the leading selector bit: each has half the length of the full vector. The control rule leaves the zero block untouched and evaluates u on the one block. It also covers a controlled scalar phase.

The rotation rule recursively evaluates the *whole axis term*. For example, `R H q` forms $c_q v + d_q H v$, and `R (C X) q` forms $c_q v + d_q [\![\texttt{C X}]\!] v$. Neither case needs a dense axis matrix. The original v must remain available until both contributions have been formed; blindly overwriting its storage while evaluating the axis would invalidate the formula.

Why branches must not be normalized. The squared norm of a vector is $v^\dagger v = \sum_j \overline{v_j} v_j$. For the phased Bell state in (6), splitting by the first qubit gives $v_0 = (h, 0)^T$ and $v_1 = (0, hz)^T$. Each branch has squared norm $1/2$, while the complete vector has norm one. These branch weights are part of the amplitudes. Normalizing each branch separately would change the state and break linearity. The same issue occurs in intermediate vectors of a rotation.

Consequently no recursive step calls a general normalization operation. If the input is normalized, unitary evolution already preserves its norm. Exact field arithmetic also permits computing inner products $v^\dagger w$ and squared norms directly. This gives useful validation without asking the scalar type to take arbitrary square roots.

Tensor layout, adjoints, and correctness

Tensor application deserves a separate derivation because its input may be entangled. In general there are no vectors x, y with $v = x \otimes y$. The algorithm must act on the full array of amplitudes. For `Tensor u t`, put $A = [\![u]\!]$, $B = [\![t]\!]$, and $D_x = 2^{n_x}$. Reshape v into the $D_u \times D_t$ array $V_{ij} = v_{iD_t+j}$. Then the Kronecker definition gives

$$\big((A \otimes B)v\big)_{iD_t+j} = \sum_{k,l} A_{ik} B_{jl} V_{kl}. \tag{11}$$

Compute it in two stages:

$$W_{ij} = \sum_l B_{jl} V_{il}, \qquad V'_{ij} = \sum_k A_{ik} W_{kj}; \qquad V' = AVB^T.$$

The transpose here is an ordinary transpose, not a conjugate transpose. It comes from arranging column-vector amplitudes into rows. Recursively apply $[\![t]\!]$ to each contiguous row, viewed as a column vector, then apply $[\![u]\!]$ to each column gathered with stride D_t. Flatten V' in the original order. These equations justify the algorithm; it never forms A, B, or $A \otimes B$.

For example, let $T = \mathrm{diag}(1, \zeta_8)$ and $v = (a, b, c, d)^T$. The row step for $H \otimes T$ gives $(a, \zeta_8 b)$ and $(c, \zeta_8 d)$; the column step gives

$$(H \otimes T)v = \frac{1}{\sqrt{2}}(a + c,\ \zeta_8(b + d),\ a - c,\ \zeta_8(b - d))^T.$$

This also illustrates why conjugating T in the row step would be wrong.

Permutations and adjoints. For `Permute ks`, scatter $v'_{f(j)} = v_j$ using the bit permutation from Section 3. For example, `Permute [1,2,0]` maps bits abc to bca, so index 4 maps to 1. A gather implementation must instead use f^{-1}. For adjoints, compile a dagger flag: reverse composition order, negate phase and rotation angles, invert permutations, and recurse through tensors, control, and direct sums. The Hermitian primitives are self-adjoint; $\mathrm{SX}^\dagger$ uses the conjugate-transposed primitive table. For a valid axis the rotation rule is $[\![\mathtt{R}\ u\ q]\!]^\dagger = [\![\mathtt{R}\ u\ (-q)]\!]$; the axis term remains u.

Proposition 2. *The ideal direct evaluator returns $w = [\![u]\!]\, v$ for every checked term u and compatible vector v. If v has cyclotomic entries, so does w. Moreover $w^\dagger w = v^\dagger v$.*

Proof. Induct over the checked expression. The structural recurrences are the actions of the corresponding block matrices. Equation (11) proves the tensor case; the primitive, permutation, and adjoint cases follow from their definitions. Closure follows from field arithmetic, and $w^\dagger w = v^\dagger [\![u]\!]^\dagger [\![u]\!]\, v = v^\dagger v$. □

The resource-bounded implementation must return this value on success, or an explicit error; the proposition does not promise that every input fits its configured limits. For G local gate applications on n qubits, the cost is $O(G2^n)$ field operations and $O(2^n)$ storage with reusable buffers. A general expression of expanded tree size s costs $O(s2^n)$, up to indexing overhead. Naive recursive rotations can retain $O(h2^n)$ live elements at nesting depth h; constant-many-vector working storage is not a bound for every expression tree.

9 Connecting the design to QPP

QPP's `HilbertSpace` already supplies scalar multiplication, addition, subtraction, and `inner`; squared norm is `inner v v`. The Matrix backend uses `CMat` for both `OpT` and `StateT`, so both inherit these operations. Statevector supplies the instance for states; its `OpT` is an application plan. The obstacle is the class's additional requirements:

```
Scalar v ~ Complex (Realnum v), Floating (Realnum v)
norm      :: v -> Realnum v
normalize :: v -> v
```

With scalars in a common cyclotomic field K, linear combinations and inner products need only field operations and conjugation:

$$av + bw \in K^D, \qquad \langle v, w\rangle = \sum_j \overline{v_j} w_j \in K, \qquad \langle v, v\rangle \in K \cap \mathbb{R}_{\geq 0}.$$

The squared norm can remain a `Cyclo`; no square root or `RealT` conversion is needed. Refactor the interface into a linear and inner-product capability with an associated scalar type, and a separate capability for norm and normalization. The exact prototype exposes checked versions of the former operations below, without claiming an instance of the current `HilbertSpace`. Cyclotomic scalars are already complex; `Complex Cyclo` and the standard `Data.Complex` arithmetic's `RealFloat` constraint are inappropriate [3]. `QOp`'s rational angles remain unchanged.

9.1 Representation and field ownership

The prototype uses Section 4's power basis; Section 13 proposes representation optimizations. An immutable private `Field` contains an index $N > 0$ and the coefficients of Φ_N; its degree is $d = \varphi(N)$. The integer polynomial coefficients use the same rational storage. A private `Coeff` is a boxed vector of exactly d rationals. Every rational is reduced, has a positive denominator, and represents zero as $0/1$. Polynomial results are reduced modulo Φ_N and padded to length d; this is coefficient normalization, not state normalization.

An exported `Cyclo` pairs a context with a `Coeff`. A shared coefficient buffer stores one context and an array of `Coeff`s; matrix and state wrappers retain their own quantum shapes. For D amplitudes the dense format stores Dd rational coordinates. Constructors are hidden: public imports check dimensions, coordinates, and limits before creating these values. The context includes the distinguished embedding $\zeta_N = e^{2\pi i/N}$; no numerical root selection occurs.

Arithmetic and promotion. Arithmetic follows the polynomial algorithms in Section 4. Euclidean remainders and their Bézout coefficients are scaled together to make each nonzero remainder monic, curbing intermediate coefficient growth. Operations expose checked functions returning `Either ExactError`; no `Floating`, `RealFloat`, or partial division instance is needed. Same-context kernels never change the field. Public binary operations first promote both operands to the least-common-multiple index; equality across contexts uses this rule too. Resource exhaustion is an error, not a false inequality.

`evalOp` chooses the index in (5). `fromAmplitudes` imports all entries into one common field. `apply` promotes the operator and input to their joint field once; `evalProg` collects all operator and input indices before execution. Matching immutable contexts are reused. Promotion uses substitution and reduction, with no automatic subfield minimization.

9.2 Evaluator interface and bounded execution

`QPP.Exact.Scalar` and `QPP.Exact.Checked` support the two backends `QPP.Semantics.Exact.Matrix` and `QPP.Semantics.Exact.Statevector`. Both backends re-export the shared `QPP.Exact.State` type and operations:

```
evalOp :: Limits -> QOp -> Either ExactError OpT
ket :: Limits -> [Bool] -> Either ExactError StateT
fromAmplitudes :: Limits -> [Cyclo] -> Either ExactError StateT
apply :: Limits -> OpT -> StateT -> Either ExactError StateT
evalProg :: Limits -> Program -> StateT
         -> Either ExactError StateT
scale :: Limits -> Cyclo -> StateT -> Either ExactError StateT
add :: Limits -> StateT -> StateT -> Either ExactError StateT
inner :: Limits -> StateT -> StateT -> Either ExactError Cyclo
```

`OpT` holds a matrix or a compiled application plan; `StateT` holds a vector without asserting unit norm. Here `normSquared limits v = inner limits v v`; subtraction uses scaling by -1 and addition. These operations check shapes and promote coefficients to a common field. Matrix linear combinations and the Frobenius inner product $\mathrm{tr}(A^\dagger B)$ use the same arithmetic, but their results need not be unitary and must not masquerade as checked `QOp` plans. The modules expose amplitude and matrix-entry accessors, and checked entrywise equality; observations return scalars with their context. `fromAmplitudes` checks a nonzero power-of-two length and field limits. A `Program` is a list of `Unitary`, `Measure`, or `Initialize` steps. Preflight checks the entire list against the input width, rejects the latter two kinds with step indices, and then executes unitaries in list order. An empty program returns a valid input directly. Direct `evalOp` compiles scalar constants once; `apply` reuses the plan or embeds its constants into the input's common field.

Limits and failures. Bound root index, field degree, coefficient slots, and integer sizes. Validation checks retained storage; each operation reserves its temporary buffers separately. Empty programs and same-field equality need no arithmetic workspace. Check dimensions such as $D^2 d$ with unbounded integers before allocation, and N before factorization. Bound integer sizes before sums/products and after reduction; conservative checks may reject later cancellation. These limits do not guarantee wall time or heap bytes. Distinguish invalid input, width mismatch, division by zero, and resource exhaustion. Invalid operators retain the checker's offending term; program width and field-degree errors identify the introducing step, counting from zero. Never approximate on failure.

9.3 Field growth: cost and ways to control it

There are three separate growth mechanisms: the number of amplitudes or tensor bonds, the field degree d, and rational coefficient bit lengths. Even an unentangled state can require large d. For distinct odd prime angle denominators $p_1, \ldots, p_k$, the sufficient-field rule can give

$$N = 8 \prod_j p_j, \qquad d = \varphi(N) = 4 \prod_j (p_j - 1).$$

Rotations with denominators 101 and 103 give $N = 83224$ and $d = 40800$. A ten-qubit statevector then has $2^{10} d = 41\,779\,200$ rational-coordinate slots in the unshared dense layout, before temporary storage and integer payloads. A product-state circuit can incur this cost: low entanglement alone does not protect dense scalar coordinates. With the straightforward polynomial algorithms, addition costs $O(d)$ and multiplication with reduction costs $O(d^2)$ rational operations; growing integer sizes add further cost. The earlier field-operation counts hide this factor.

The common field is sufficient, not always necessary for each value. Cancellation can yield a rational number in a huge ambient field. Conversely, for distinct primes p, q, the number $\zeta_p \zeta_q = \zeta_{pq}^{p+q}$ is primitive of order pq, since $p + q$ is coprime to pq. Its degree really is $(p-1)(q-1)$. A compact symbolic expression can delay expanded coordinates, but cannot make that algebraic degree disappear. We propose the following distinct approaches; none guarantees efficient evaluation of every program.

Reduce unnecessary field extensions. Reduce rational angles and use actual root orders: ζ_{2b}^a has order $2b/\gcd(|a|, 2b)$, and ζ_{4b}^a has order $4b/\gcd(|a|, 4b)$. Validated, phase-preserving rewrites can cancel inverse gates or combine rotations about the same axis before field selection. Global phases must remain: rotations are generally periodic in q with period 4, not 2. Collect roots after such rewrites; do not bypass axis validation.

Store fewer coordinates and reuse arithmetic. Sparse reduced polynomials store only nonzero coordinates; special forms for zero, rationals, and roots of unity avoid large mostly empty arrays. Cache field contexts, reduction data, and repeated embeddings. Basis-state unitary evaluation stays in D_N, permitting integer numerators with power-of-two denominators rather than general rational arithmetic. Sparse products or reduction may nevertheless become dense, and MPS elimination needs general division.

Delay promotion and expansion. Per-coefficient or per-core fields can postpone a common-field embedding until values interact; exact subfield descent can shrink results after cancellation. Both require additional bookkeeping and exact conversion checks. Factored states, MPS, or shared scalar expressions may retain products instead of expanding them. For example, product-state cores can keep unrelated angle fields separate. Contraction, inner products, or equality may eventually force them together. Small bonds alone are insufficient if every core still uses a large common-field dense basis. These are optimization directions beyond the first common-field prototype.

9.4 Proof obligations and implementation evidence

The propositions establish the ideal algorithms. The code must maintain the context/coordinate invariant and return the specified result on success. Tests provide evidence, not a proof: backend agreement can miss a shared sign or wire-order error.

The 84 `ExactTest` tests cover scalar laws, embedding composition, independent polynomial identities, and dense/direct agreement. Expected-value cases include $[\![\texttt{R Z 1}]\!] = -iZ$, $[\![\texttt{R (Id 0) 2}]\!] = [-1]$, $[\![\texttt{Phase (1/2)}]\!] = [i]$, the two worked examples, and non-self-inverse wire permutations. Further cases cover inverse circuits, unequal tensor widths, direct sums, QFTs through three qubits against the analytic Fourier matrix, composite axes, rewrites, and input/resource errors. Randomized properties use exact equality and preserve valid shapes when shrinking failures. Degree-48 inversion cases exercise coefficient growth. All eight repository suites pass (645 tests). Run `cabal test all`; `cabal run ExactEvaluation` reproduces the phased Bell example. Approximate export is for observation only, never equality or pivoting.

10 What exact MPS evaluation can and cannot mean

A statevector has 2^n amplitudes even when its structure is simple. For $n \geq 1$, a matrix product state (MPS) stores those amplitudes as products of small matrices:

$$v_{b_1 \cdots b_n} = A^{[1]}_{b_1} A^{[2]}_{b_2} \cdots A^{[n]}_{b_n}, \qquad A^{[j]}_b \in K^{\chi_{j-1} \times \chi_j}, \quad \chi_0 = \chi_n = 1. \tag{12}$$

The matrices $A^{[j]}_b$ are the cores, $b \in \{0, 1\}$ is the physical qubit index, and χ_j is a bond dimension. Site j here corresponds to QPP wire $j - 1$. Multiplying the selected matrices gives one scalar amplitude. If every bond is at most χ, the storage is $O(n\chi^2)$. Product states have bond dimension one; entanglement may require larger bonds, potentially exponential in n.

At width zero, store one scalar $z \in K$. The existing `Phase q` is already the required scalar operator: it sends z to $e^{i\pi q} z$. No new syntax is needed. Tensoring a zero-qubit value with a nonempty MPS scales one core; two zero-qubit values multiply. Its squared norm is $\bar{z}z$. This explicit base case also represents zero, whereas an empty product of cores alone would always give one.

Unfolding a state across a cut. Group the first k bits into a row index and the remaining bits into a column index. This reshapes the amplitudes into a matrix $M^{(k)} \in K^{2^k \times 2^{n-k}}$. An MPS factors this matrix through a space of dimension χ_k, so rank $M^{(k)} \leq \chi_k$. For a nonzero state, the minimum possible bond dimension at each cut is this rank. The Schmidt decomposition obtains the same rank from an SVD:

$$M^{(k)} = U \Sigma V^\dagger, \qquad |v\rangle = \sum_{\alpha=1}^{r} \sigma_\alpha |\ell_\alpha\rangle \otimes |r_\alpha\rangle.$$

The left and right Schmidt vectors are orthonormal, and σ_α^2 are the nonzero eigenvalues of $(M^{(k)})^\dagger M^{(k)}$ [5]. The left vectors are columns of U; the right vectors are the complex conjugates of columns of V. SVD is attractive numerically because it supplies orthonormal bases and controlled approximation by discarding small singular values.

The operational consequence: preserve the state, replace SVD. Cyclotomic entries do not guarantee cyclotomic singular values. For the integer matrix

$$M = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix}, \qquad \det(xI - M^T M) = x^3 - 8x^2 + 15x - 4,$$

the squared singular values are not cyclotomic. Consequently neither are the singular values: squaring a cyclotomic number would stay cyclotomic. The optional argument below proves this claim.

This is an obstruction for actual states, not just arbitrary matrices. The sum of squared entries of M is 8. Padding $M/\sqrt{8}$ to 4×4 gives the amplitudes of a normalized four-qubit state whose Schmidt values are noncyclotomic. It is QOp-preparable: a Hadamard and two Hadamards controlled on its one branch give nonzero amplitudes $(2, 1, 1, 1, 1)/\sqrt{8}$. A reversible computational-basis permutation places them in the required entries. Such permutations can be built from X and multiply controlled X; they are more general than QPP's wire-reordering `Permute`.

The state itself still has cyclotomic amplitudes. Therefore exact MPS representation is possible, while SVD-based canonicalization cannot in general stay in cyclotomic arithmetic. QPP's existing numerical `Exact` MPS setting means no truncation; it still uses floating-point SVD and should not be confused with the algebraic exactness proposed here.

Optional proof of the obstruction (Galois theory). This paragraph uses splitting fields and the Galois correspondence; it is not needed to implement the algorithms. The displayed cubic has no rational root and discriminant $916 = 4 \cdot 229$, a nonsquare. It is therefore irreducible with Galois group S_3, so adjoining one root gives a non-Galois cubic field. In contrast, every field between $\mathbb{Q}$ and a cyclotomic field is Galois: the cyclotomic Galois group is abelian, hence every subgroup is normal [1]. No root of this cubic can belong to a cyclotomic field. Dividing its positive square roots by $\sqrt{8}$, itself cyclotomic, does not remove the obstruction for the normalized state.

Rank factorization and exact local updates

We need a factorization that removes linear dependence without finding orthonormal bases. Let $T \in K^{m \times n}$ have rank $r > 0$. Choose r independent columns with indices J and form $B = T_{:,J}$. Every column of T is a unique linear combination of columns of B, so $T = BC$ for some $C \in K^{r \times n}$. Choose r independent rows of B, indexed by I; then

$$T = BC, \qquad B = T_{:,J}, \qquad C = (B_{I,:})^{-1} T_{I,:}. \tag{13}$$

The invertible $r \times r$ matrix $B_{I,:}$ makes the coefficients computable using field arithmetic alone. For example,

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 \end{pmatrix};$$

the factorization retains the one-dimensional column space and requires no singular vectors or normalization.

Implement the inverse action by elimination, not by forming an explicit inverse. Select pivots by exact nonzero tests and handle the zero matrix separately. The factors stay in K, although division may leave the smaller dyadic ring D_N. Rank is unchanged on extending K to $\mathbb{C}$: it is the largest order of a nonzero minor, and those minors have the same zero/nonzero status in either field. Successively factor the unfoldings of a nonzero tensor to obtain an exact MPS with minimal cut ranks; the rank-based existence argument underlying tensor trains does not require Schmidt values [6].

Applying a local gate. Contract two neighboring cores, apply the two-qubit gate matrix U, and regard the result as a matrix:

$$T_{(\alpha,s),(t,\beta)} = \sum_{a,b,\gamma} U_{st,ab} A_a^{[j]}(\alpha, \gamma) A_b^{[j+1]}(\gamma, \beta), \qquad s, t, a, b \in \{0, 1\}. \tag{14}$$

The indices α, γ, β are bonds. Factor $T = BC$ and reshape the factors into replacement cores. With neighboring bonds at most χ, this costs $O(\chi^3)$ field operations. A one-qubit gate only mixes the two physical slices of one core. Adjacent SWAPs can route a nonlocal gate to neighboring sites.

These cores need not be orthonormal. A change $A_b^{[j]} \mapsto A_b^{[j]} G$ and $A_b^{[j+1]} \mapsto G^{-1} A_b^{[j+1]}$ leaves amplitudes unchanged for invertible G; this freedom is called a gauge. Compute squared norms by contracting an environment through the cores:

$$E_0 = [1], \qquad E_j = \sum_{b=0}^{1} (A_b^{[j]})^\dagger E_{j-1} A_b^{[j]}, \qquad E_n = [\, v^\dagger v \,].$$

One core alone generally does not contain the state's norm.

A local rank factorization removes dependence in its local matrix. It need not give globally minimal bonds if surrounding block bases are redundant; that requires justified rank-revealing sweeps. No tolerance or singular-value truncation is involved. Bond dimensions, field degree, and rational coefficient sizes all remain independent sources of cost. Exactness does not guarantee bounded entanglement or an efficient representation for every circuit.

11 HQP project: an exact MPS interpreter in four weeks

Question and starting point. Can rank factorization replace SVD in a useful exact MPS simulator? Supply the tested scalar module, exact Matrix/Statevector oracles, circuit corpus, and a compiler from the supported `QOp` fragment to positioned gates with validated wire indices. Scalar arithmetic and front-end development are outside this project. Prerequisites are Haskell, tensor indexing, Gaussian elimination, and the circuit model.

Required scope. Implement opaque MPS cores over the supplied field with checked bond dimensions; basis-state construction; exact contraction; one-site and adjacent two-site unitary updates; and rank factorization. Support X, Y, Z, H, SX, rational rotations about X, Y, Z, global phase, CNOT, controlled rational phase, and adjacent SWAP. The supplied compiler flattens composition and tensor padding. General nested control, arbitrary `DirectSum`, and composite axes must report unsupported input.

Week	Work	Demonstrable result
1	Understand the scalar API; implement exact rank factorization, including rectangular, rank-deficient, and zero matrices.	Tests verify $T = BC$, full factor rank, and deterministic pivot behavior.
2	MPS cores, basis states, contraction, and one-/two-site updates. Compare amplitudes after every gate.	Bell and GHZ preparation and inverse circuits agree exactly with the statevector oracle.
3	Gate-interface integration, adjacent SWAPs, environment contraction, and bounded randomized tests.	The supported fragment has explicit errors and exact differential tests; squared norms are computed without SVD assumptions.
4	Reproducible experiments, code cleanup, report, and demonstration.	Explain measured costs using bonds, field degree, coefficient growth, and circuit structure.

Acceptance criteria and deliverables. Deliver an integrated Haskell module, tests, benchmark driver, and a 6–10-page report. On bounded circuits up to eight qubits, compare intermediate amplitudes, squared norms, and inverse round trips with the exact oracle. Cover entanglement creation/removal, rank deficiency, cancellation, non-dyadic angles, controlled phases, and SWAP ordering.

Vary depth and denominators separately on product, GHZ, and shallow entangling circuits. Report time, available memory measurements, bonds, field degree, coefficient bits, and resource failures; no speedup is required.

Scope control. Require correct local rank reduction, not globally minimal bonds. General `QOp` semantics, measurement, SVD, approximation, and scalar-library development are excluded. Rank-compression sweeps or nonadjacent-gate routing are stretch work.

12 MSc thesis: exact tensor-network semantics and coefficient growth

Research aim. Study the cost of exact MPS/MPO evaluation. A matrix product operator (MPO) represents an operator by cores with input/output bits and two bond indices; contracting bonds reconstructs its matrix entries. Connect proved invariants to measured arithmetic behavior.

Three research questions.

1. When does MPS/MPO evaluation use less time or memory than exact statevector evaluation, and which representation choices determine this?
2. Which pivot and compression policies control coefficient growth? Does smaller bond dimension imply faster exact evaluation?
3. How do entanglement, denominators, and program structure constrain performance and usefulness as a compiler-validation oracle?

Core technical work. Define gauge and wire-order invariants. Exact representations already exist by closure and rank factorization; a complete compositional interpreter, proved against the dense semantics without dense fallback, is the engineering baseline. Use MPO arithmetic or direct branching; with $P_b = |b\rangle\langle b|$,

$$[\![\mathtt{C}\ u]\!] = P_0 \otimes I_{2^{n_u}} + P_1 \otimes [\![u]\!],$$
$$[\![\mathtt{DirectSum}\ u\ t]\!] = P_0 \otimes [\![u]\!] + P_1 \otimes [\![t]\!] \quad (n_u = n_t).$$

Rotations use (8). Exact MPS addition may concatenate or block-diagonalize bonds, followed by rank-based compression. Handle zero branches explicitly.

Prove rank-revealing sweeps that remove redundant block bases in both directions; minimal-rank existence alone does not establish their correctness. Choose two comparisons: first-nonzero versus coefficient-size pivots; eager versus bond-size-triggered compression; common versus local fields. Fraction-free or modular elimination is optional.

Period	Milestone
Month 1	Literature review, reproduce the local prototype, define semantics and experiment protocol.
Months 2–3	General compositional evaluator and rank-sweep invariants; exact agreement tests; complete a baseline implementation.
Month 4	Implement the selected arithmetic/compression variants; instrument field operations, bonds, and coefficient sizes.
Month 5	Controlled experiments and compiler-rewrite case studies; explain successes and failures.
Month 6	Consolidate proofs and artifacts, write the thesis, and prepare a reproducible release and presentation.

Evaluation and success. Use exact Matrix/Statevector as correctness oracles, uncompressed MPS as a compression baseline, and floating MPS for performance/accuracy comparison. Vary bonds and field degrees separately on product states, QFTs, Clifford+T, mixed-denominator circuits, and rewrite regressions. Publish seeds, limits, builds, results, and timeouts. Success means correctness and reproducible evidence explaining costs, without universal speedup. Measurement and parallel arithmetic are later extensions.

13 Student project: efficient cyclotomic arithmetic

An independent four-week project builds on the supplied prototype scalar module and evaluator clients: which storage and arithmetic choices reduce the cost of exact evaluation at a fixed mathematical result?

Require two improvements: sparse reduced polynomials with a measured sparse/dense switching rule, and cached field contexts and embeddings. Preserve the scalar API, distinguished complex embedding, and exact equality, including inversion and general rational coefficients for rank factorization. Minimal-field discovery is not required.

Week 1 instruments and tests the baseline; week 2 implements sparse arithmetic; week 3 adds caching and integrates both representations; week 4 evaluates and documents. Compare exact results against the prototype on bounded random values, roots of unity, cancellations, mixed-denominator circuits, and elimination traces.

Include conjugation, inversion, promotion, and cross-field equality; Walck's library is a possible independent comparison [2].

Report setup and evaluation time, available peak-memory measurements, field degree, sparsity, coefficient bits, and resource failures. Deliver an integrated module, tests, benchmark driver, and short report. Use the measurements to motivate extensions such as local fields, certified descent, dyadic arithmetic, or faster multiplication.

14 Boundaries, related work, and the implementation decision

Cyclotomic arithmetic supports exact amplitudes, matrix entries, inner products, squared norms, and Born probabilities. It does not support arbitrary normalization or SVD. For example, the cyclotomic vector $(1, 2 + \sqrt{2})$ has squared norm $7 + 4\sqrt{2}$, whose square root is not cyclotomic. Normalizing this vector would leave cyclotomic arithmetic; unitary evaluation avoids the issue by preserving the input norm.

Measurement need not leave cyclotomic arithmetic: unnormalized projected vectors remain cyclotomic, with probabilities given by ratios of exact squared norms. For an outcome projector P and density matrix ρ with entries in the same cyclotomic field, conditioning gives $P\rho P^\dagger/\mathrm{tr}(P\rho P^\dagger)$ for a nonzero probability, which also stays in the field. Sampling requires ordering and randomness; these extensions are outside the prototype.

Optional proof of the normalization boundary (Galois theory). Put $F = \mathbb{Q}(\sqrt{2})$, $a = 7 + 4\sqrt{2}$, and $a' = 7 - 4\sqrt{2}$. The field norm, the product of the two conjugates in F, gives $aa' = 17$. Hence neither is a square in F: the norm of a square would be a rational square. For $\sqrt{a'}$ to lie in $F(\sqrt{a})$, $a'/a = (a')^2/17$ would have to be a square in F, but $\sqrt{17} \notin F$. Thus $\mathbb{Q}(\sqrt{a}) = F(\sqrt{a})$ lacks a conjugate of $\sqrt{a}$ and is not Galois. The criterion in Section 10 excludes $\sqrt{a}$ from the cyclotomics, and its reciprocal cannot be cyclotomic either.

Cyclotomic field theory [1] and Walck's library [2] provide foundations and comparison. Exact Clifford+T synthesis also uses the dyadic cyclotomic ring [4]; here the task is evaluation. MPS, tensor trains, and MPO arithmetic provide established constructions [5, 6, 7]. The student projects investigate their exact cyclotomic costs in QPP, using the prototype as a reference.

References

[1] J. S. Milne. *Fields and Galois Theory*, version 5.10, 2022. Sections on Galois correspondence and cyclotomic extensions. https://www.jmilne.org/math/CourseNotes/FT.pdf.

[2] S. N. Walck. *cyclotomic: A subfield of the complex numbers for exact calculation*, Haskell package, version 1.1.2, 2023. https://hackage.haskell.org/package/cyclotomic-1.1.2.

[3] GHC libraries. *Data.Complex*, `base-4.21.0.0` documentation. https://hackage.haskell.org/package/base-4.21.0.0/docs/Data-Complex.html.

[4] B. Giles and P. Selinger. Exact synthesis of multiqubit Clifford+T circuits. *Physical Review A* 87, 032332, 2013. https://doi.org/10.1103/PhysRevA.87.032332.

[5] U. Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics* 326, 96–192, 2011. https://doi.org/10.1016/j.aop.2010.09.012.

[6] I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing* 33(5), 2295–2317, 2011. https://doi.org/10.1137/090752286.

[7] C. Hubig, I. P. McCulloch, and U. Schollwöck. Generic construction of efficient matrix product operators. *Physical Review B* 95, 035129, 2017. https://doi.org/10.1103/PhysRevB.95.035129.